

# Reviewer Pack — Minimal Number of Binary Quadratic Forms and Resolution Proo...

Entry 849957120937 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-04 10:19:17 UTC

## Minimal Number of Binary Quadratic Forms and Resolution Proof Width

**Entry ID:** 849957120937 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-04 10:19:17 UTC

### 1. Conjecture

**Field A** (mathematical branch): Algebraic Combinatorics (Binary Quadratic Forms) **Field B** (complexity object): Resolution Proof Complexity

#### Statement:

For any given CNF  $\varphi$ , the resolution proof width  $w(\varphi)$  is linearly related to the minimal number of binary quadratic forms required to encode the clauses of  $\varphi$ , such that  $w(\varphi) = \Theta(n^{1/2})$ , where  $n$  is the number of variables in  $\varphi$ .

#### Rationale (proposer's reasoning):

Binary quadratic forms have been used in cryptography and coding theory but are rarely applied in complexity theory. Their algebraic structure could provide a novel encoding for resolution proofs, potentially revealing connections between their properties and proof complexity. If true, this conjecture would suggest that the hardness of resolution proofs is related to the geometric properties of binary quadratic forms.

**Taxonomy category:** binary\_quadratic\_forms\_resolution\_proof\_complexity (status at proposal time: )

### 2. Pre-registration (Popper-style)

**Hash (SHA-256 prefix):** 62bc54f566084057

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

#### Acceptance criterion:

The conjecture is supported if for all 30 seeds, the ratio of resolution proof width to the square root of the number of variables is within  $\pm 20\%$  (i.e.,  $0.8 \leq w(\varphi)/\sqrt{n} \leq 1.2$ ) and the count of distinct quadratic forms used across all clauses is equal to or less than 1.5 times the resolution proof width, while falsified if either condition is not met.

### 3. Barrier filter (F1)

**Final:** PASS

| Barrier        | Verdict | Confidence | LLM <sub>1</sub> | LLM <sub>2</sub> |
|----------------|---------|------------|------------------|------------------|
| RELATIVIZATION | SAFE    | 1.00       | SAFE             | UNCERTAIN        |
| NATURAL_PROOFS | SAFE    | 0.50       | UNCERTAIN        | UNCERTAIN        |
| ALGEBRIZATION  | SAFE    | 0.50       | UNCERTAIN        | UNCERTAIN        |
| KARP_LIPTON    | SAFE    | 0.50       | UNCERTAIN        | UNCERTAIN        |

### 4. Novelty audit

**Verdict:** NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

### 5. Empirical test harness

**Multi-seed protocol:** 5 seeds (11, 23, 37, 53, 71)

**Sandbox:** isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=-9, elapsed=204.9s

#### 5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n):
        clauses = []
        for _ in range(2 * n):
            clause = [random.randint(-n, -1), random.randint(1, n)]
            clauses.append(clause)
        return clauses

    def resolution_width(cnf):
        stack = cnf[:]
        while True:
            new_clause = None
            for i in range(len(stack)):
                for j in range(i + 1, len(stack)):
                    if -stack[i][0] in stack[j]:
                        new_clause = [x for x in stack[i] if x != -stack[i][0]] + [y for y in stack[j] if y != -stack[i][0]]
                        break
                if new_clause:
                    break
            if not new_clause:
                return len(stack)
            stack.append(new_clause)

    n = random.randint(5, 40)
    cnf = generate_cnf(n)
    width = resolution_width(cnf)

    metric_name = "resolution_width"
```

```

metric_value = width
instances_tested = 1
n_max = n
conjecture_holds = False
counterexample = ""

if width <= math.sqrt(n) * 1.2 and width >= math.sqrt(n) * 0.8:
    conjecture_holds = True

return {
    "metric_name": metric_name,
    "metric_value": metric_value,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17, 19,

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    if all(r["conjecture_holds"] for r in results):
        support_fraction = 1.0
    else:
        support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={sum(r['metric_value'] for r in results) / len(results)} st
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")

```

## 6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

## 7. Test stdout (last 2KB)

(empty)

## 8. Critic adversarial review

**Critic verdict:** CHALLENGE

**Critic reasoning:**

skipped: test crashed before producing data

## 9. Final verdict & safety rail

**Verdict:** INCONCLUSIVE

## Reasoning:

The test crashed before producing data, which means that neither the support nor falsification conditions could be evaluated. | next: Re-run the test with a different set of seeds or under different conditions to ensure it completes successfully and provides the necessary data for evaluation.

## 11. Audit log (LLM calls)

**Total LLM calls:** 10

| #  | Phase           | Provider      | Model            | Tokens in | out | Latency (ms) |
|----|-----------------|---------------|------------------|-----------|-----|--------------|
| 1  | propose         | ollama_remote | glm4:latest      | 0         | 0   | 14452        |
| 2  | propose         | ollama_remote | glm4:latest      | 0         | 0   | 12895        |
| 3  | preregistration | ollama_remote | glm4:latest      | 0         | 0   | 14619        |
| 4  | novelty         | ollama_remote | glm4:latest      | 0         | 0   | 8349         |
| 5  | novelty         | ollama_remote | glm4:latest      | 0         | 0   | 8544         |
| 6  | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 16223        |
| 7  | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 29260        |
| 8  | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 14888        |
| 9  | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 10616        |
| 10 | judge           | ollama_remote | glm4:latest      | 0         | 0   | 56494        |

**Totals:** 0 input tokens, 0 output tokens, ~\$0.0000 cost, 186340 ms total latency. Provider mix: {'ollama\_remote': 10}

(full prompt+response transcripts available in `research/audit/849957120937.jsonl`)

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/849957120937.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file:** `research/pvsnp_sandbox/test_*.py` (per-cycle)

**Replay tarball:** `research/replay/849957120937.tar.gz` (if generated)